

Scalable Parallel-in-Time Integration for Equations of Motion: Particle Production in Analog Cosmologies

Nathaniel Chapman¹

¹Department of Computer Science, Central Washington University

March 16, 2025

Abstract

Simulating time-dependent physics has traditionally been constrained to using sequential algorithms, thus not benefiting from advances in parallel computing. Parallel-in-time integration attempts to address this limitation with methods such as the Parareal algorithm. As the performance of the Parareal algorithm scales with the number of processors, it is well-suited to use the massively-parallel nature of graphics processing units. Additional performance gains are seen when the physics is wave-like, as using a spectral method allows for each node in a distributed system to evaluate the Parareal algorithm. Particle production in different cosmologies is used to highlight the performance gains from these methods.

*This work is dedicated to
my friends for sharing laughs and rants,
Mr. Chris Lacey for making physics phun,
Dr. Brandon Peden for showing me how to be a physicist,
and Dr. Andy Piacsek for making sure I finish this damn thing.*

Contents

1. Introduction	4
2. Background	4
2.1. Parallel-in-Time Integration (PinT)	4
2.1.1. Parareal	4
2.1.2. Multigrid Reduction in Time (MGRIT)	4
2.1.3. Parallel Full Approximation Scheme in Space and Time (PFASST)	4
2.2. High-Performance Computing	4
2.2.1. Multi-threading & GPU Computing	4
2.2.2. Multi-processing & Distributed Computing	4
2.3. Equations of Motion	4
2.3.1. Differential Equations	4
2.3.2. Traditional Numerical Methods	4
2.4. Analog Cosmology	5
2.4.1. BEC Analogs of FLRW Cosmologies	6
2.4.2. Variable Speed of Sound and Inflation	6
2.4.3. The Field Equation	6
2.4.4. Phononic & Free Particle Modes	7
2.4.5. Initial Conditions	8
2.4.6. Particle Production	8
3. Methods	9
3.1. The Parareal Algorithm	9
3.1.1. Subproblem Preparation	9
3.1.2. Parallel Propagation	10
3.1.3. Corrections	10
3.1.4. Iteration & Convergence	10
3.2. The Parareal Algorithm at Scale	10
3.2.1. The Parareal Algorithm on the GPU	12
3.2.2. The Parareal Algorithm on Multiple GPUs	14
4. Discussion	15
4.1. Numerical Analysis	15
4.1.1. Convergence	15
4.1.2. Stability	15
4.1.3. Error	15
4.2. Algorithm Analysis	15
4.2.1. Time Complexity	15
4.2.2. Space Complexity	15
4.3. Benchmarks	15
5. Particle Production in Analog Cosmologies	15
6. Conclusion	15
6.1. Future work	15
Bibliography	16

1. Introduction

2. Background

2.1. Parallel-in-Time Integration (PinT)

There are 3 traditional ways to parallelize the solution of a computational problem: CPU parallelization, GPU parallelization, and Distributed computing. While CPU parallelization is more straightforward to implement, GPU parallelization can allow for runtimes to decrease by many orders of magnitudes. Even lower run times can be achieved by combining either of these parallelization schemes with running them on multiple machines. This investigation focuses on parallelizing the solution of equations of motion using GPUs and multiple machines.

These approaches can offer massive increases in performance, but only for problems that are well-posed to be parallelized. Traditionally, initial value problems have been unable to be parallelized due their dependance on causality. Several methods have been created to overcome this limitation. These methods include the Parareal algorithm, Multigrid Reduction in Time (MGRIT), Parallel Full Approximation Scheme in Space and Time (PFASST). This investigation focuses on the Parareal algorithm.

2.1.1. Parareal

- The Parareal method has mostly been applied to first-order ordinary differential equations.
- Part of the novelty of this work is that it focuses on building support for second-order ODES

2.1.2. Multigrid Reduction in Time (MGRIT)

2.1.3. Parallel Full Approximation Scheme in Space and Time (PFASST)

2.2. High-Performance Computing

Some key aspects of high-performance computing (HPC) are:

2.2.1. Multi-threading & GPU Computing

2.2.2. Multi-processing & Distributed Computing

2.3. Equations of Motion

2.3.1. Differential Equations

- Ordinary Differential Equations (ODE)
- Systems of ODEs e.g. N-Body
 - uncoupled
 - coupled
- Partial Differential Equations (PDE) e.g. wave equation

2.3.2. Traditional Numerical Methods

- ODEs
 - Symplectic Integration
 - Traditional Methods in solving Equations of Motion
 - Do not use Runge-Kutta methods
 - Symplectic Euler
 - Velocity Verlet
 - etc.
- PDEs
 - Method of Lines
 - Method of Relaxation

One of the most important algorithms used in evolving equations of motion is the velocity Verlet method.

2.4. Analog Cosmology

Directly measuring the properties of the universe just after the Big Bang is impossible, as that was almost 14 billion years ago. Even *indirectly* measuring these properties is extremely difficult via traditional means. During these brief moments just after the Big Bang, the universe expanded rapidly in a particular way. During this expansion there were particles popping in and out of existence, each with its own dynamics (e.g. position and momentum).

What is less difficult is cooling down gases to near absolute-zero (about a billion times colder than empty space). Gases made of certain atoms or molecules have properties that can be changed to almost any value we want. Because of this, we can turn our knobs in the lab to make the gas behave in a way that matches a certain mathematical model.

For some types of gases, we can choose how strongly the particles in the gas interact with each other. It turns out that we can choose a certain interaction strength so that the mathematical model that describes how the gas behaves **exactly** matches the mathematical model of how particles are produced in the universe in the moments just after the Big Bang! When this happens, we can call this ultra-cold gas an “analog universe”.

Because of this mathematical equality, we can effectively observe the particles in the moments just after the Big Bang, but in the lab. Moreover, these observations can be made using a tried-and-true system that has been developed and used over the past thirty years[1].

Studies have investigated gases with

- phonons with a temporal frequency that is nonlinearly related to the spatial frequency (dispersion relation) [2–4]
- analog massive particles [5–7]
- analog particles with non-zero spin [8–11]
- analog universes with different expansion behavior [12–15]
- analog universes with non-zero background velocity [16–20]
- promising experimental realizations [15,21–23]

Computationally, simulations have had to investigate analog universes of reduced dimension due to the unreasonable time it takes to simulate full-dimensional systems without using high-performance computing. These simulations have been deemed accurate enough as it is proposed that a full-dimensional simulation would yield qualitatively similar results. In order to be confident in the level of accuracy of the reduced-dimensional simulations, those results need to be compared to those of a full-dimensional simulation. This will not only allow a quantification of the error in the reduced-dimensional simulation, but also a measure of the error-to-resource efficiency of a full-dimensional simulation.

Future insight into these analog systems and the fundamental properties of the early universe require computational support. A readily-usable computational model will not only allow theoretical investigations to make predictions, but also allow experimental research to have a guide on where to go next and have something with which to compare. The availability of this work is paramount to more efficient and more physically-accurate insight into our universe.

I choose a gas with linear dispersion, analog massless and spin-0 particles, an analog universe undergoing a de Sitter expansion with zero background velocity. The de Sitter spacetime is chosen because of its significance to modern cosmology and previously predicted particle production (as done by Hawking) [24–26]. Particle production will be calculated with standard methods [27,28]. The numerical parameters chosen in this study also follow from previous studies [29]. Computational implementations will be done using high-performance methods.

2.4.1. BEC Analogs of FLRW Cosmologies

The gas as is outlined is a ground-state Bose-Einstein condensate (BEC) without thermal or quantum fluctuations. The dynamics of the BEC are described by the Gross-Pitaevskii equation (GPE) under a Bogoliubov mean-field approximation (also known as the *nonlinear Schrödinger equation*),

$$i\hbar\partial_t\psi(t, \vec{x}) = \left[-\frac{\hbar^2}{2m}\nabla^2 + V_{\text{ext}}(x) + U|\psi(t, \vec{x})|^2 \right] \psi(t, \vec{x}). \quad (1)$$

The wave function ψ can be expanded using a linearized Madelung density-phase representation

$$\psi(t, \vec{x}) = \sqrt{n_0 + \Delta n(t, \vec{x})} e^{i(\theta_0 + \Delta\theta(t, \vec{x}))}, \quad (2)$$

where $n_0 + \Delta n(t, \vec{x})$ and $\theta_0 + \Delta\theta(t, \vec{x})$ are the linearized forms of the real density and phase fields, respectively. Furthermore, $\Delta n(t, \vec{x})$ and $\Delta\theta(t, \vec{x})$ are the density and phase perturbations, respectively.

With the Madelung representation defined in Eq. 2, the GPE in Eq. 1 becomes [30]

$$\frac{1}{\sqrt{-g}}\partial_\mu[\sqrt{-g}g^{\mu\nu}\partial_\nu\Delta\theta] = 0, \quad (3)$$

where

$$g_{\mu\nu} = \left(\frac{n_0}{c}\right)^{\frac{2}{d-1}} \begin{pmatrix} -c^2 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \delta_{ij} \end{pmatrix}. \quad (4)$$

Eq. 4 can be interpreted as the covariant metric tensor (with determinant g) describing an analog, spatially flat, Friedmann-Lemaître-Robertson-Walker universe, c is the speed of sound in the condensate, and d is the number of spatial dimensions. Eq. 3 describes both the phase-perturbations $\Delta\theta$ of oscillations with low spatial frequency (i.e. low momentum phonons) in the BEC and also the dynamics of a quantum field that produces massless, spin-0 particles.

2.4.2. Variable Speed of Sound and Inflation

The time dependence of the system is completely captured in the speed of sound by

$$c(t)^2 = U(t)\frac{n_0}{m} = 4\pi\frac{\hbar^2}{m^2}n_0\ell(t), \quad (5)$$

with atoms of mass m , scattering length $\ell(t)$, and number density n_0 . With the dimensionless scaling function $a(t)$, the interaction strength $U(t)$ (or equivalently the scattering length) has time dependence defined by

$$U(t) \equiv U_0 a(t), \quad (6)$$

where $U_0 = U(t_0)$ for an initial time t_0 . Then Eq. 5 becomes

$$c(t) = c_0 \sqrt{a(t)}. \quad (7)$$

This time dependence of the speed of sound and interaction strength allows us to draw another analogy. If the speed of sound decreases with time, it would appear as if the space between the source and the observer were increasing. Similarly, if the speed of sound increases with time, it would appear as if the space between the source and observer is decreasing. With this in mind, if the interaction strength between atoms $U(t)$ decreases with t , the analog universe is expanding, and if $U(t)$ increases, the analog universe is contracting.

2.4.3. The Field Equation

With the scaling function, Eq. 3 becomes¹

¹ $\frac{\dot{a}(t)}{a(t)}$ is the Hubble parameter for an expanding universe with scaling parameter $a(t)$.

$$\text{2D Field Equation: } \partial^2 \Delta\theta - \frac{3}{2} \frac{\dot{a}(t)}{a(t)} \partial_t \Delta\theta - c_0^2 a(t) \nabla^2 \Delta\theta = 0 \quad (8)$$

$$\text{3D Field Equation: } \partial^2 \Delta\theta - \frac{\dot{a}(t)}{a(t)} \partial_t \Delta\theta - c_0^2 a(t) \nabla^2 \Delta\theta = 0 \quad (9)$$

These equations only differ in the constant coefficient to the first derivative i.e. the dissipative term. If we consider the *conformal time* η defined by $d\eta = \sqrt{a(t)}dt$, the two-dimensional field equation (Eq. 8) becomes

$$\partial_\eta^2 \Delta\theta - \frac{1}{2} \frac{\dot{a}(\eta)}{a(\eta)} \partial_\eta \Delta\theta - c_0^2 \nabla^2 \Delta\theta = 0. \quad (10)$$

Eq. 10 governs the dynamics of oscillations in the BEC for a time-dependent speed of sound, and the behavior of the massless, spin-0 particles in an expanding universe.

2.4.4. Phononic & Free Particle Modes

The phase $\Delta\theta(t, \vec{x})$ and density $\Delta n(t, \vec{x})$ perturbation fields can be expanded in a Fourier plane-wave amplitude representation as

$$\Delta\theta(t, \vec{x}) = \frac{1}{\sqrt{V}} \sum_{\vec{k}} e^{i\vec{k} \cdot \vec{x}} \tilde{\theta}_{\vec{k}}(t) \hat{b}_{\vec{k}}(t) + e^{-i\vec{k} \cdot \vec{x}} \tilde{\theta}_{\vec{k}}^*(t) \hat{b}_{\vec{k}}^\dagger(t) \quad (11)$$

$$\Delta n(t, \vec{x}) = \frac{1}{\sqrt{V}} \sum_{\vec{k}} e^{i\vec{k} \cdot \vec{x}} \tilde{n}_{\vec{k}}(t) \hat{b}_{\vec{k}}(t) + e^{-i\vec{k} \cdot \vec{x}} \tilde{n}_{\vec{k}}^*(t) \hat{b}_{\vec{k}}^\dagger(t) \quad (12)$$

where $\vec{k}$ is the wave-vector of the phase and density perturbations, $\tilde{\theta}$ is the discrete Fourier Transform of the phase perturbation, $\hat{b}$ and $\hat{b}^\dagger$ are the annihilation and creation operators, respectively. From this, there arises two domains to consider²: phononic quasiparticles with linear dispersion, and free-particle-like quasiparticles with quadratic dispersion.

The boundary between phononic and free-particle-like quasiparticles is described by the critical wave-number k_c . The critical wave-number k_c is defined [30] in terms of the healing length of the BEC ξ , where

$$\xi(t) = \frac{\hbar}{\sqrt{2mc}} = \frac{\xi_0}{\sqrt{a(t)}}, \quad (13)$$

$$k_c(t) = \frac{1}{\xi(t)} = \frac{\sqrt{a(t)}}{\xi_0} \quad (14)$$

Wave-vectors $\vec{k}$ with wave-number $k = \|\vec{k}\|$ such that $k \ll k_c$ are called *phononic*³. Likewise, those with $k \gg k_c$ are called *free-particle-like*⁴.

For phononic quasiparticles, the analog between a BEC and an inflationary cosmology is exact. Including free-particle-like quasiparticles forces the analog to include “trans-Planckian” effects and analog Lorentz violation[30]. For these reasons, this investigation focuses only on phononic quasiparticles; namely wave-vectors $\vec{k}$ such that $\|\vec{k}\| \ll k_c$. Additionally, the free-particle-like regime has nonlinear dispersion which significantly increases the complexity of the field equation[30].

Only considering phononic modes, the two-dimensional field (Eq. 10) then becomes

$$\partial_\eta^2 \tilde{\theta}_{\vec{k}} - \frac{1}{2} \frac{\dot{a}(\eta)}{a(\eta)} \partial_\eta \tilde{\theta}_{\vec{k}} - c_0^2 k^2 \tilde{\theta}_{\vec{k}} = 0. \quad (15)$$

²These domains can also be described as “acoustic” and “Bogoliubov”, respectively.

³Phononic quasiparticles have linear dispersion $\omega = ck$ characterized by neglecting quantum pressure.

⁴Free-particle-like quasiparticles have quadratic dispersion characterized by $\omega_k(t)^2 = \frac{k^2}{2m} \left(\frac{\hbar^2 k^2}{2m} + 2U(t)n_0 \right)$ characterized by including quantum-pressure

For a de Sitter universe, the scaling function is defined such that $a(t) = e^{-t/t_s}$. Therefore, the field equation for a two-dimensional de Sitter universe is

$$\partial_\eta^2 \tilde{\theta}_{\vec{k}} - \frac{1}{\eta} \partial_\eta \tilde{\theta}_{\vec{k}} + c_0^2 k^2 \tilde{\theta}_{\vec{k}} = 0, \quad (16)$$

2.4.5. Initial Conditions

As the phase and density perturbations need to be continuous at $t = 0$ when the expansion begins, the initial conditions i.e. spacetime boundary conditions are defined by this physical conservation. Therefore

$$\tilde{\theta}_{\vec{k}}(t = 0) = \tilde{\theta}_{\vec{k}0} \quad (17)$$

$$\partial_t \tilde{\theta}_{\vec{k}}(t = 0) = -\frac{U}{\hbar} \tilde{n}_{\vec{k}0} \quad (18)$$

where the initial derivative of the phase perturbation's Fourier amplitude (Eq. 18) is determined by neglecting the quantum pressure[30].

2.4.6. Particle Production

The number of particles $N_{\vec{k}}(t)$ produced at time t with wave-vector $\vec{k}$ is described by the equation

$$N_{\vec{k}(t)} = |u_{\vec{k}}^{\text{out}*}(t) v_{\vec{k}}^{\text{exp}*}(t) - v_{\vec{k}}^{\text{out}*}(t) u_{\vec{k}}^{\text{exp}*}(t)|^2 \quad (19)$$

where

$$u_{\vec{k}}(t) = \frac{1}{2\sqrt{n_0}} \tilde{n}_{\vec{k}}(t) + i\sqrt{n_0} \tilde{\theta}_{\vec{k}}(t) \quad (20)$$

$$v_{\vec{k}}(t) = \frac{1}{2\sqrt{n_0}} \tilde{n}_{\vec{k}}(t) - i\sqrt{n_0} \tilde{\theta}_{\vec{k}}(t), \quad (21)$$

“out” and “exp” corresponding to regions of spacetime outside of and inside the expansion of the universe.

To summarize, to calculate the number of particles produced at position $\vec{x}$ and time t with wave-vector $\vec{k}$ looks like the following:

- 1) Solve the field equation (Eq. 15) for $\tilde{\theta}_{\vec{k}}(t)$
- 2) Differentiate $\tilde{\theta}_{\vec{k}}(t)$ to get $\tilde{n}_{\vec{k}}(t)$ as in equation (Eq. 18)
- 3) Combine $\tilde{\theta}_{\vec{k}}(t)$ and $\tilde{n}_{\vec{k}}(t)$ as in equations (Eq. 20)
- 4) Calculate the number of particles at time t with wave-vector $\vec{k}$ as in Eq. 19

3. Methods

Things covered in this chapter:

- It's about runtime!
- The Parareal Algorithm
 - Subproblem interpretation
 - Leads to recursive implementation
 - The Main steps of the Parareal Algorithm:
 - Subproblem preparation
 - Parallel Propagation
 - Sequential Corrections
 - Iteration & Convergence
- The Parareal Algorithm at Scale
 - The Parareal Algorithm on the GPU
 - MORE CORES -> MORE FAST
 - The Parareal Algorithm on Multiple GPUs
 - Automatic cluster topology identification

3.1. The Parareal Algorithm

The Parareal Algorithm aimed to solve the problem of physics taking too long to simulate; it didn't.

The main idea of the Parareal algorithm (PA) is to break up a single IVP into many smaller IVPs using some low-accuracy solution, solve those in parallel using high-accuracy methods, correct your initial solution using the sub-solutions, then make a new low-accuracy solution based on the corrected data, and repeat this process until the solution doesn't change. The end result of this procedure is a solution identical to one produced by directly using the high-accuracy method **CITATION NEEDED**) while taking a less time. Because the PA wraps traditional (sequential) solvers, it could be considered a “meta-” or “higher-order” method to solve IVPs.

3.1.1. Subproblem Preparation

Given an initial value problem P :

$$P = \{\mathcal{L}(t, u, \partial_t u, \partial_t^2 u) = f(t), \quad u(0) = u_0, \partial_t u(0) = v_0, \quad [T, T + \Delta T]\} \quad (22)$$

where $D = [T, T + \Delta T]$. Choose the number of subproblems N (suggested: the number of compute cores). Partition the time domain D into subdomains D_p :

$$D_p = \left[T + \frac{p}{N} \Delta T, T + \frac{p+1}{N} \Delta T \right] = [T_p, T_{p+1}]. \quad (23)$$

Use a coarse propagator $\mathcal{C}_0$ to compute initial solutions $\{u_p^0\}_p, \{v_p^0\}_p$ defined such that

$$\{u_p^0\}_p = \{u_0^0, u_1^0, u_2^0, \dots, u_{N-1}^0\} \quad (24)$$

$$\{v_p^0\}_p = \{v_0^0, v_1^0, v_2^0, \dots, v_{N-1}^0\} \quad (25)$$

where the superscript denotes that this is the zeroth-iteration

Algorithm 1: PREPARE THE SUBPROBLEMS

INPUT: Second order initial value problem P , Coarse solver C
OUTPUT: Solution for P made from `pos_seq` and `vel_seq`, and array of subproblems `subproblems`
// choose discretization
1 $N \leftarrow$ number of subproblems *// e.g. multiple of # of computer cores*
// partition the domain of P into N subdomains e.g. $[a, b] \rightarrow \{[a, c], [c, b]\}$
2 `subdomains` $\leftarrow$ `partition(P.domain)`
// use coarse solver to get positions and velocities for the root problem
3 `pos_seq, vel_seq` $\leftarrow$ `propagate(P, C)`
// create subproblems
4 **for** i from 1 to N
5 `subdomain` $\leftarrow$ i -th domain partition `subdomains[i]`
6 `pos0` $\leftarrow$ initial position for i -th subproblem `pos_seq[i]`
7 `vel0` $\leftarrow$ initial velocity for i -th subproblem `vel_seq[i]`
8 `subproblems[i]` $\leftarrow$ ivp on `subdomain` with initial values `pos0` and `vel0` for acceleration $P.\text{acc}$
9 **return** Solution for root problem with `pos_seq` and `vel_seq`, and array of subproblems `subproblems`

Algorithm 1: Prepare the subproblems

3.1.2. Parallel Propagation

Use a coarse propagator $\mathcal{C}$ (e.g., Symplectic-Euler with a large time step) and a fine propagator $\mathcal{F}$ (e.g. Velocity-Verlet with a small time step). Solve each subproblem p in parallel.

3.1.3. Corrections

Compute corrections for the coarse solutions using:

$$\eta_p^i = \mathcal{F}u_p^i(T_{p+1}) - \mathcal{C}u_p^i(T_{p+1}), \quad (26)$$

and apply corrections sequentially:

$$u_{\{p+1\}}^i(T_{p+1}) = u_p^i(T_{p+1}) + \eta_p^i. \quad (27)$$

3.1.4. Iteration & Convergence

Repeat the process for updated initial values until convergence, e.g.:

$$|u_p^i - u_p^{\{i-1\}}| < \varepsilon \quad \forall p \leq N. \quad (28)$$

In addition to parallelizing, part of the magic of the Parareal algorithm lies in solving each subproblem not once, but twice with different solves or *propagators*. The next step in the process is to choose *coarse*, and *fine*. It should be noted that this coarse propagator does not need to be the same as the coarse propagator that was chosen in preparing the subproblems. Possible propagators include the semi-implicit Euler method with a large time step for the coarse propagator, and the velocity-verlet method with a small time step for the fine propagator; in order to satisfy energy-conservation, symplectic integrators should be used. Without loss of generality, let the chosen coarse and fine propagators be denoted $\mathcal{C}, \mathcal{F}$, respectively, and the n_s -th data point for propagator $\mathcal{S}$ in the p -th subproblem at iteration i be denoted $u_{pn_s}^i$ and defined traditionally by $u_{pn_s}^i = \mathcal{S}u_{pn_s-1}^i$, and the solution from propagator $\mathcal{S}$ on subproblem p is the ordered collection of points $\mathcal{S}u_p^i = u_{pn_s}^{i_{n_s}}$.

3.2. The Parareal Algorithm at Scale

- Parallelize on the GPU instead of the CPU
 - Port functionality to kernel calls

- Technical: Instead of objects with properties, it’s just the same index for a bunch of different arrays
 - Vibes: Makes logic less understandable comp
- 1) From the host, launch the Parareal kernel on the device
 - 1) Each core on the device executes the same sequence of instructions (kernel), but uses different thread-local variables such as their thread id, block id, etc.
 - 2) Use traditional and sequential solvers on each core for each subproblem
 - 3) Write the final point to an array
 - 2) Send solution data back to host for sequential correction
 - 3) Host corrects and loops
- Distribute problems over multiple processes/devices
 - Preparing the Cluster
 - Currently only works for an ssh-cluster i.e. a collection of machines that can all be accessed via ssh from the head node
 - A “remote node” could also be a single process on a single machine, the differences are straightforward
 - Given a head node and a collection of remote nodes
 - 1) Spawn a worker, or “sub-manager”, process on each remote node
 - 2) Each sub-manager identifies how many devices are available to the node, and send that information back to the manager process
 - 3) The manager process spawns a worker process on the appropriate node for each device on that node
 - 4) Each process acquires a device
- 1) For each problem:
 - 1) Send it to a process
 - 2) Execute the parareal algorithm on that problem using the assigned device
 - 3) Send the result to the manager process to be used in corrections

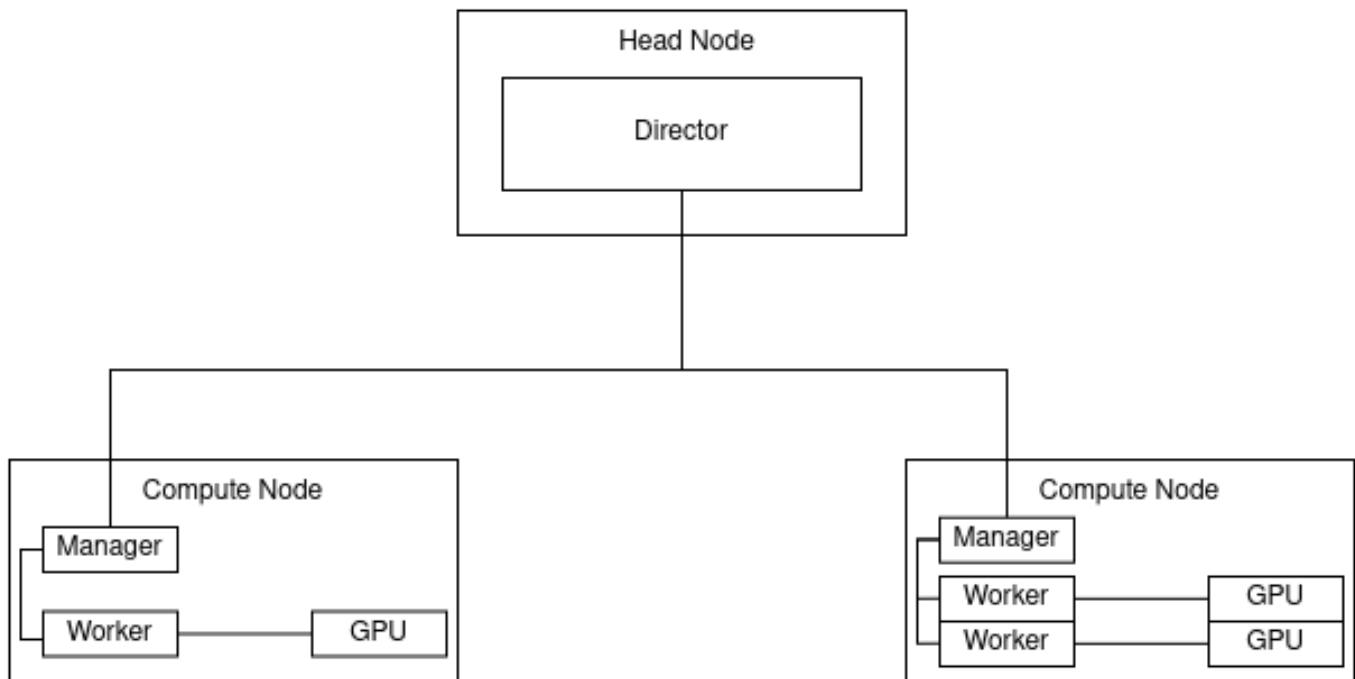


Figure 1: A representative cluster topology.

3.2.1. The Parareal Algorithm on the GPU

- Execute the parallel propagation on the GPU
 - Requires transforming “regular code” into a “kernel” that’s evaluated on every computer core
 - Make solution data an array that is copied to the device which can then just write to the appropriate index
 - Use thread-local variables (e.g. `threadIdx.x`, `blockIdx.x`, etc.) to identify the appropriate index
 - “Don’t overwrite your neighbor” by index striding.

Algorithm 2: DISCRETIZE_KERNEL

INPUT: Sequence of points in the subdomain `dompnts`

OUTPUT: Nothing

```

1 npnts ← number of points in dompnts
2 lb ← lower bound of this subdomain dompnts[1]
3 ub ← upper bound of this subdomain dompnts[-1]
4 step ← (ub - lb) / npnts
5 for i from 2 to (npnts - 1)
6   L dompnts[i] ← lb + (i - 1) * step
    
```

Algorithm 2: GPU kernel to calculate the discretized points in a subdomain in-place

Algorithm 3: PROPAGATE_KERNEL

INPUT: Solver function `sol`, Acceleration function `acc`, Domain points `dompnts`, Position sequence `pos_seq`, Velocity sequence `vel_seq`

OUTPUT: Nothing

```

1 npnts ← number of points in domain dompnts
2 for i from 2 to (npnts - 1)
3   op ← old position pos_seq[i - 1]
4   ov ← old velocity vel_seq[i - 1]
5   np ← new position pos_seq[i]
6   nv ← new velocity vel_seq[i]
7   L np, nv ← solver(op, ov, acc, step)
    
```

Algorithm 3: GPU kernel to propagate solutions in-place

Algorithm 4: KERNEL

INPUT: Solver function `sol`, Acceleration function `acc`, Sequence of sequences of domain points `dss`, Sequence of sequences of positions `pss`, Sequence of sequences of velocities `vss`

OUTPUT: Nothing

```

1 nsols  $\leftarrow$  number of subproblems or subdomains
  // INDEX STRIDING
2 index  $\leftarrow$  (block_id - 1) * block_dim + thread_id
3 stride  $\leftarrow$  grid_dim * block_dim
4 for i from index to nsols in steps of stride
    // DISCRETIZE DOMAINS
5     dompnts  $\leftarrow$  sequence of points in this subdomain dss[i]
6     discretize_kernel(dompnts)
    // ALLOCATE SOLUTIONS
7     pos_seq  $\leftarrow$  sequence of positions for this subproblem pss[i]
8     vel_seq  $\leftarrow$  sequence of velocities for this subproblem vss[i]
9     propagate_kernel(sol, acc, dompnts, pos_seq, vel_seq)

```

Algorithm 4: Main kernel

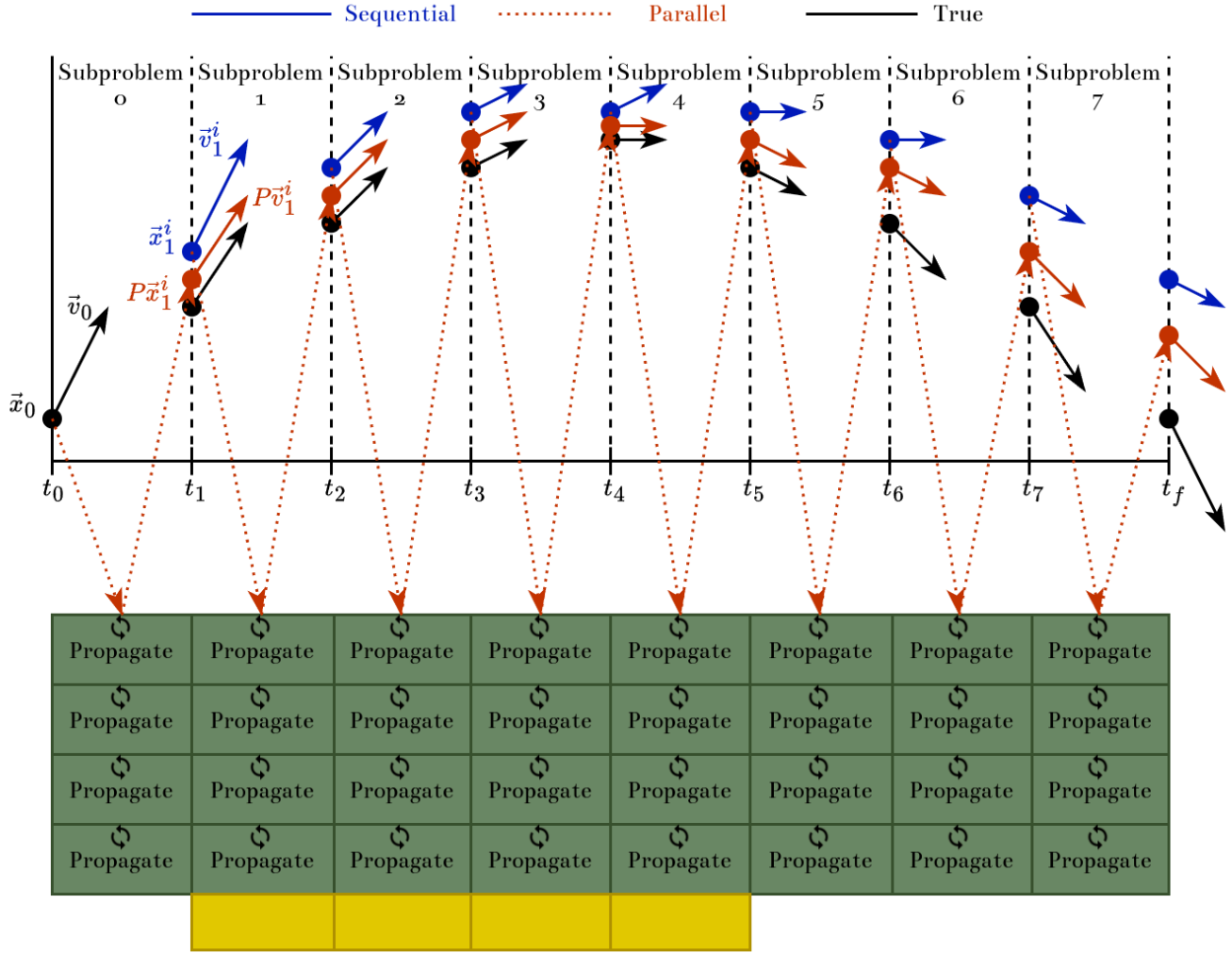


Figure 2: Sequential solutions (blue) are sent to the GPU to be finely-propagated (red) in parallel; true solutions (black) are shown for comparison.

3.2.2. The Parareal Algorithm on Multiple GPUs

- How did I glue GPU and Distributed computing together with the Parareal algorithm to make it scalable?
 - GPU Computing: solve each subproblem on a each gpu core
 - make stuff into arrays
 - thread indexing blocks streaming multiprocessors
 - Distributed Computing: solve each root problem on each node
 - “Just add another machine”
 - Automatically determine number of devices on each node
 - Sharing data across processes

4. Discussion

4.1. Numerical Analysis

- use the pendulum i.e. the simple harmonic oscillator to test numerical analysis properties since we know the analytic solutions and can compare.

4.1.1. Convergence

4.1.2. Stability

4.1.3. Error

4.2. Algorithm Analysis

4.2.1. Time Complexity

4.2.2. Space Complexity

4.3. Benchmarks

5. Particle Production in Analog Cosmologies

- Solve the partial differential equation
- spectral decomposition
- system of equations $\partial_t^2 \tilde{\theta} - \left(\frac{\dot{a}}{a}\right) \partial_t \tilde{\theta} - ac^2 k^2 \tilde{\theta} = 0$ for wavenumber $k \leq k_c$

6. Conclusion

- Equations of motion can now benefit from parallel solvers.
- Certain problems are well-suited to a divide-and-conquer approach.
- Problems with “doubly parallel” characteristics can leverage both local and distributed parallelism, achieving significant computational efficiency.
- These advancements pave the way for modeling acoustics in expanding volumes.

6.1. Future work

- Krylov enhanced subspaces
- CUDA dynamic parallelism
- Implement with C, Fortran, CUDA, NVSHMEM, MPI
- Make gpu-backend-agnostic with KernelAbstractions.jl
- this physics could be better done with PFASST

Bibliography

- [1] M. H. Anderson, J. R. Ensher, M. R. Matthews, C. E. Wieman, and E. A. Cornell, Observation of Bose-Einstein Condensation in a Dilute Atomic Vapor, *Science* **269**, 198 (1995).
- [2] C. Barceló, S. Liberati, and M. Visser, Analogue Gravity, *Living Reviews in Relativity* **8**, (2005).
- [3] (n.d.).
- [4] S. Weinfurtner, S. Liberati, and M. Visser, Modelling Planck-scale Lorentz violation via analogue models, *Journal of Physics: Conference Series* **33**, 373 (2006).
- [5] S. Liberati, M. Visser, and S. Weinfurtner, Analogue quantum gravity phenomenology from a two-component Bose-Einstein condensate, *Classical and Quantum Gravity* **23**, 3129 (2006).
- [6] S. Weinfurtner, S. Liberati, and M. Visser, Analogue model for quantum gravity phenomenology, *Journal of Physics A: Mathematical and General* **39**, 6807 (2006).
- [7] S. Weinfurtner, S. Liberati, and M. Visser, *Analogue Space-Time Based on 2-Component Bose-Einstein Condensates*, in *Quantum Analogues: From Phase Transitions to Black Holes and Cosmology*, edited by W. G. Unruh and R. Schützhold (Springer Berlin Heidelberg, Berlin, Heidelberg, 2007), pp. 115–163.
- [8] S. Liberati, M. Visser, and S. Weinfurtner, Naturalness in an Emergent Analogue Spacetime, *Phys. Rev. Lett.* **96**, 151301 (2006).
- [9] M. Visser and S. Weinfurtner, Massive phonon modes from a BEC-based analog model, (2004).
- [10] M. Visser and S. Weinfurtner, Massive Klein-Gordon equation from a Bose-Einstein-condensation-based analogue spacetime, *Phys. Rev. D* **72**, 44020 (2005).
- [11] R. Schützhold, G. Plunien, and G. Soff, Dielectric Black Hole Analogs, *Phys. Rev. Lett.* **88**, 61101 (2002).
- [12] T. Jacobson, Introduction to Quantum Fields in Curved Spacetime and the Hawking Effect, (2004).
- [13] M. Visser, Some general bounds for one-dimensional scattering, *Phys. Rev. A* **59**, 427 (1999).
- [14] P. J. Steinhardt and N. Turok, Cosmic evolution in a cyclic universe, *Phys. Rev. D* **65**, 126003 (2002).
- [15] C. Barceló, S. Liberati, and M. Visser, Probing semiclassical analog gravity in Bose-Einstein condensates with widely tunable interactions, *Phys. Rev. A* **68**, 53613 (2003).
- [16] S. Weinfurtner, Analogue model for an expanding universe, *General Relativity and Gravitation* **37**, 1549 (2005).
- [17] P. O. Fedichev and U. R. Fischer, Gibbons-Hawking Effect in the Sonic de Sitter Space-Time of an Expanding Bose-Einstein-Condensed Gas, *Phys. Rev. Lett.* **91**, 240407 (2003).
- [18] P. O. Fedichev and U. R. Fischer, Observer dependence for the phonon content of the sound field living on the effective curved space-time background of a Bose-Einstein condensate, *Phys. Rev. D* **69**, 64021 (2004).
- [19] P. O. Fedichev and U. R. Fischer, “Cosmological” quasiparticle production in harmonically trapped superfluid gases, *Phys. Rev. A* **69**, 33602 (2004).
- [20] M. Uhlmann, Y. Xu, and R. Schützhold, Aspects of cosmic inflation in expanding Bose-Einstein condensates, *New Journal of Physics* **7**, 248 (2005).
- [21] J. L. Roberts, N. R. Claussen, J. P. Burke, C. H. Greene, E. A. Cornell, and C. E. Wieman, Resonant Magnetic Field Control of Elastic Scattering in Cold ^{85}Rb , *Phys. Rev. Lett.* **81**, 5109 (1998).
- [22] S. L. Cornish, N. R. Claussen, J. L. Roberts, E. A. Cornell, and C. E. Wieman, Stable ^{85}Rb Bose-Einstein Condensates with Widely Tunable Interactions, *Phys. Rev. Lett.* **85**, 1795 (2000).

-
- [23] J. M. Vogels, C. C. Tsai, R. S. Freeland, S. J. J. M. F. Kokkelmans, B. J. Verhaar, and D. J. Heinzen, Prediction of Feshbach resonances in collisions of ultracold rubidium atoms, *Phys. Rev. A* **56**, R1067 (1997).
 - [24] A. H. Guth, Inflationary universe: A possible solution to the horizon and flatness problems, *Phys. Rev. D* **23**, 347 (1981).
 - [25] G. W. Gibbons and S. W. Hawking, Cosmological event horizons, thermodynamics, and particle creation, *Phys. Rev. D* **15**, 2738 (1977).
 - [26] A. S. Lapedes, Bogoliubov transformations, propagators, and the Hawking effect, *Journal of Mathematical Physics* **19**, 2289 (2008).
 - [27] N. D. Birrell and P. C. W. Davies, *Quantum Fields in Curved Space* (Cambridge University Press, 1982).
 - [28] S. A. Fulling, *Aspects of Quantum Field Theory in Curved Spacetime* (Cambridge University Press, 1989).
 - [29] C. W. Gardiner, J. R. Anglin, and T. I. A. Fudge, The stochastic Gross-Pitaevskii equation, *Journal of Physics B: Atomic, Molecular and Optical Physics* **35**, 1555 (2002).
 - [30] P. Jain, S. Weinfurter, M. Visser, and C. W. Gardiner, Analog model of a Friedmann-Robertson-Walker universe in Bose-Einstein condensates: Application of the classical field method, *Phys. Rev. A* **76**, 33616 (2007).
 - [31] S. G. Johnson, QuadGK.jl: Gauss-Kronrod integration in Julia, (2013).
 - [32] E. W. Weisstein, Gauss-Kronrod Quadrature, (n.d.).
 - [33] J. Li, General explicit difference formulas for numerical differentiation, *Journal of Computational and Applied Mathematics* **183**, 29 (2005).
 - [34] H. Nguyen and R. Tsai, A stable parareal-like method for the second order wave equation, *Journal of Computational Physics* **405**, 109156 (2020).
 - [35] X. Dai and Y. Maday, Stable Parareal in Time Method for First- and Second-Order Hyperbolic Systems, *SIAM Journal on Scientific Computing* **35**, A52 (2013).
 - [36] M. J. Gander and S. Vandewalle, Analysis of the Parareal Time-Parallel Time-Integration Method, *SIAM Journal on Scientific Computing* **29**, 556 (2007).
 - [37] M. J. Gander and S. Güttel, PARAEXP: A Parallel Integrator for Linear Initial-Value Problems, *SIAM Journal on Scientific Computing* **35**, C123 (2013).